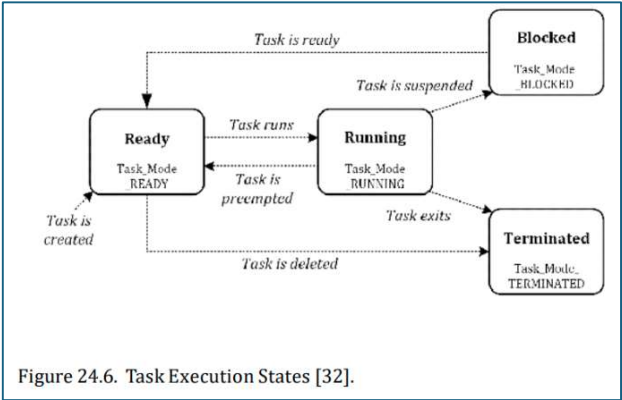


Q1 – TI RTOS Question



Q4.1 – GPIO DriverLib Coding Question

PxDIR <bitwise cmd> <Pin # as BIT#>	PxREN <bitwise cmd> <Pin # as BIT#>	PxOUT <bitwise cmd> <Pin # as BIT#>
PxDIR sets Pin as either Input or Output	PxREN enables or disables Resistor	PxOUT sets I/O Behavior
(Output (1)) PxDIR = BIT#;	N/A	(Low Output (0)) PxOUT &= BIT#;
(Input (0)) PxDIR &= BIT#;	PxREN = BIT#;	(High Output (1)) PxOUT = BIT#;
(Input (0)) PxDIR &= BIT#;	PxREN = BIT#;	(Pull Down Input (0)) PxOUT &= BIT#;
★ BIT# = Pin #, Pin 0 = BIT0		(Pull Up Input (1)) PxOUT = BIT#;

DriverLib GPIO

- GPIO_setAsOutputPin
- GPIO_setAsInputPin
- GPIO_setAsInputPinWithPullDownResistor
- GPIO_setAsInputPinWithPullUpResistor
- GPIO_setAsPeripheralModuleFunctionOutputPin
- GPIO_setAsPeripheralModuleFunctionInputPin
- GPIO_enableInterrupt
- GPIO_clearInterruptFlag
- GPIO_getEnabledInterruptStatus
- GPIO_setOutputHighOnPin
- GPIO_setOutputLowOnPin
- GPIO_toggleOutputOnPin

DriverLib UART

- EUSCIA_UART_initParam
- EUSCIA_UART_init
- EUSCIA_UART_enable
- EUSCIA_UART_enableInterrupt
- EUSCIA_UART_getInterruptStatus
- EUSCIA_UART_transmitData
- EUSCIA_UART_receiveData

Q2 – Python Question

Operation	Result
x in s	True if an item of s is equal to x, else False
x not in s	False if an item of s equal to x, else True
s + t	The concatenation of s and t
s * n, n * s	n shallow copies of s concatenated
s[i]	ith item of s, origin 0
s[i:j]	Slice of s from i to j
s[i:j:k]	Slice of s from i to j with step k
len(s)	Length of s
min(s)	Smallest item of s

Q3 – TA0CCR1 Value Estimation Question

```
// Headers are not shown but properly defined
volatile unsigned int tcap=0, kp=0;
int main(void) {
    WDTCTL = WDTPW | WDTHOLD;
    PM5CTL0 &= ~LOCKLPM5;
    PDIR |= BIT3;
    PDIR &= ~BIT0;
    PSEL1 &= ~BIT0;
    PSEL0 |= BIT0;
    TA0CTL = TASSEL_2 | MC_2 | TACLRL;
    TA0CTL = CM_2 | CCIS_0 | CCIE | CAP | SC5;
    enable_interrupt();
    while (1) {
        POUT=0x00;
        TA0CTL |= TACLRL;
        delay_cycles(2000);
        while (kp<5) {
            POUT^=BIT3;
            delay_cycles(2000);
            kp++;
        }
        while(1) {
            // no operation();
            _delay_cycles(1000);
        }
    }
    return 0;
}
#pragma vector = TIMER0_A1_VECTOR
__interrupt void Timer0_A1_ISR(void) {
    if((TA0CCR1L & CCIFG)!=0) {
        tcap = TA0CCR1L;
        TA0CCR1L &= ~CCIFG;
    }
}
```

Step 1: Determine measurement type from TA0CCR1L
CM_1 - Rising Edge
CM_2 - Falling Edge (ex.)
CM_3 - Both

Step 2: Determine number of periods and duration (ex. kp<5, so ~4 periods) (ex. 2000 ticks long)

Step 3: Calculate TA0CCR1 ticks based on TA0CCR1L by graphing it. Graph starts on LOW because of the TACLRL (ex. 4 periods of falling edge)

Falling edge stops at 4, so sum all the ticks before 4. 2000+2000+2000+2000 = 8000

Q4.2 – GPIO DriverLib Debugging Question

```
#include <msp430.h>
#include "driverlib.h"
int main (void) {
    WDT_A_hold(WDT_A_BASE); // hold the watchdog timer
    PMM_unlockLPM5(); // clear LOCKLPM5 bit
    GPIO_setAsOutputPin(GPIO_PORT_P1, GPIO_PIN0); // output (P1.0)
    GPIO_setAsOutputPin(GPIO_PORT_P1, GPIO_PIN1); // output (P1.1)
    GPIO_setAsInputPinWithPullUpResistor(GPIO_PORT_P5, GPIO_PIN6);
    // input, pull-up resistor (P5.6)
    GPIO_clearInterrupt(GPIO_PORT_P5, GPIO_PIN6); // clear interrupt flag (P5.6)
    GPIO_enableInterrupt(GPIO_PORT_P5, GPIO_PIN6); // enable interrupt (P5.6)
    _enable_interrupt(); // enable general interrupt
    while (1) {
        GPIO_toggleOutputOnPin(GPIO_PORT_P1, GPIO_PIN0); // toggle (P1.0)
        _delay_cycles(250000); // delay
    }
    return 0;
}

#pragma vector=PORT5_VECTOR
__interrupt void Port5_ISR_handler(void) {
    uint32_t status;
    status = GPIO_getInterruptStatus(GPIO_PORT_P5, GPIO_PIN6);
    // read interrupt status (P5.6)

    if(status) { // check whether the PIN is set
        GPIO_toggleOutputOnPin(GPIO_PORT_P1, GPIO_PIN1); // toggle (P1.1)
    }
    GPIO_clearInterrupt(GPIO_PORT_P5, GPIO_PIN6); // clear port1 interrupt flag
}
```

Program 23.1-LED will be turned on or off as the button is pressed or released by a user.

Q5 – RTOS + Duty Cycle Question

Period = 1 / Frequency
Period = $T_{on} + T_{off}$
Duty Cycle = $T_{on} / (T_{on} + T_{off}) * 100$
(On Percentage)

```
// Headers and user driver configuration are not shown but properly defined
void task1Fxn(UArg arg0) {
    while (1) {
        Semaphore_pend(semHandle, BIOS_WAIT_FOREVER);
        GPIO_write(Board_LED0, Board_LED_ON);
        Task_sleep((UInt)arg0);
        Semaphore_post(semHandle);
    }
}

void task2Fxn(UArg arg0) {
    while (1) {
        Semaphore_pend(semHandle, BIOS_WAIT_FOREVER);
        GPIO_write(Board_LED0, Board_LED_OFF);
        GPIO_toggle(Board_LED1);
        Task_sleep((UInt)arg0);
        Semaphore_post(semHandle);
    }
}

int main(void) {
    Task_Params task1Params, task2Params;
    Semaphore_Params semParams;
    Board_initGeneral();
    Board_initGPIO();
    GPIO_Write(Board_LED0, Board_LED_OFF);
    GPIO_write(Board_LED1, Board_LED_OFF);
    Task_Params_init(&task1Params);
    task1Params.arg0 = 100;
    task1Params.stackSize = TASKSTACKSIZE;
    task1Params.stack = &task1Stack;
    task1Params.priority = 1;
    Task_construct(&task1Struct, (Task_FuncPtr)task1Fxn, &task1Params, NULL);
    Task_Params_init(&task2Params);
    task2Params.arg0 = 250;
    task2Params.stackSize = TASKSTACKSIZE;
    task2Params.stack = &task2Stack;
    task2Params.priority = 1;
    Task_construct(&task2Struct, (Task_FuncPtr)task2Fxn, &task2Params, NULL);
    Semaphore_Params_init(&semParams);
    Semaphore_construct(&semStruct, 1, &semParams);
    semHandle = Semaphore_handle(&semStruct);
    BIOS_start();
    return 0;
}
```

(1) Task 1 has priority. Turns LED0 On for 100 Ticks.

(2) Task 2 goes next. Turns LED0 OFF, Toggles LED1 for 250 Ticks. This means both LEDs will blink.

Task 1 is on for 100 ticks and off for 250 ticks. Task 2 is on for 250 ticks after Task 1, so at 350 ticks. Task 2 turns off and on every 350 ticks, giving an on cycle of 350, off cycle of 350.

Task 1 Duty Cycle: $100/(100+250)*100 = 28.6\%$
Task 2 Duty Cycle: $350/(350+350)*100 = 50\%$

Q6 – RP Pico Duty Cycle Questions

Period (T) = $\frac{1}{\text{pwm.freq(n)}}$ sec	Operator	Description
On Time (T) = $\frac{\text{pwm.duty.bitres(n)}}{2^{\text{bitres}}}$	&	bitwise AND
		bitwise OR
	^	bitwise exclusive OR
	<<	shift left
	>>	shift right
Duty Cycle = Period * On Time	~	one's complement

```
from sense_emu import SenseHat

sense = SenseHat()
CL1=(0,0,0)
CL2=(255,255,255)
sense.clear(CL1)
while True:
    sense.show_message("Hello world",text_colour=CL2,back_colour=CL1)
```

```
from sense_emu import SenseHat
from time import sleep

sense = SenseHat()
CL1=(0,0,0)
CL2=(255,255,255)
sense.clear(CL1)
while True:
    sense.show_letter("E",CL2)
    sleep(0.5)
    sense.show_letter("S",CL2)
    sleep(0.5)
    sense.show_letter("E",CL2)
    sleep(0.5)
    sense.show_letter("T",CL2)
    sleep(0.5)
    sense.show_letter("3",CL2)
    sleep(0.5)
    sense.show_letter("6",CL2)
    sleep(0.5)
    sense.show_letter("9",CL2)
    sleep(0.5)
```

```
from sense_emu import SenseHat
from time import sleep

sense = SenseHat()
CL1=(0,0,0)
CL2=(255,255,255)
sense.clear(CL1)
while True:
    sense.set_pixel(2,2,CL2)
    sense.set_pixel(4,2,CL2)
    sense.set_pixel(3,4,CL2)
    sense.set_pixel(1,5,CL2)
    sense.set_pixel(2,6,CL2)
    sense.set_pixel(3,6,CL2)
    sense.set_pixel(4,6,CL2)
    sense.set_pixel(5,5,CL2)
    sleep(1)
    sense.set_pixel(3,6,CL1)
    sense.set_pixel(3,7,CL2)
    sleep(1)
    sense.set_pixel(3,7,CL1)
```

GPIO, Button and LED:

```
from machine import Pin
from utime import sleep
led=Pin(25,Pin.OUT)
button=Pin(15,Pin.IN)
while True:
    if button.value()--0:
        led.value(1)
        sleep(0.5)
        led1.value(0)
        sleep(0.5)
```

```
from machine import Pin
from utime import sleep
led=Pin(25,Pin.OUT)
while True:
    led.value(1)
    sleep(0.5)
    led.value(0)
    sleep(0.5)
```

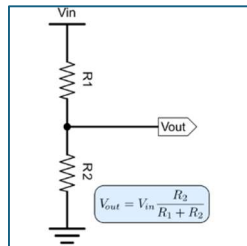
ADC Example:

```
import machine as pico
from utime import sleep
adc1=pico.ADC(0)
while True:
    adc_raw=adc1.read_u16()
    print(adc_raw)
    sleep(0.5)
```

$$\text{ADC Res} = 2^n$$
$$\text{Unsigned ADC} = 2^n \left(\frac{V_{in} - V_R}{V_R - V_R} \right)$$
$$\frac{V}{A} = \frac{3.3}{4095} = \frac{V_{range}}{\text{Resolution}}$$
$$T(^{\circ}\text{C}) = (ADC_{raw} - NADC_{Low}) \left(\frac{T_{high} - T_{low}}{NADC_H - NADC_L} \right) + 30$$

Temperature Sensor Example:

```
import machine as pico
from utime import sleep
adc1=pico.ADC(4)
while True:
    adc_raw=adc1.read_u16()
    adc_conv=adc_raw*3.3/65535
    tempC=27-(adc_conv-0.706)/0.001721
    print(tempC)
    sleep(0.5)
```



PWM (LED):

```
from machine import Pin, PWM
from utime import sleep
pwm=PWM(Pin(25))
pwm.freq(50)
pwm.duty_u16(0)
while True:
    for PWMvalue in range(0,65535,2000):
        pwm.duty_u16(PWMvalue)
        sleep(0.1)
    for PWMvalue in range(65535,0,-2000):
        pwm.duty_u16(PWMvalue)
        sleep(0.1)
```



Chapter 25 – Open Source Dev

A sketch is a program that Arduino uses. It contains two functions which are setup() and loop(). The code in the “setup” function will be executed once, and the code in the “loop” function will be executed repeatedly.

```
const int button1 = PUSH1; // push button (left)
const int button2 = PUSH2; // push button (right)
const int led1 = RED_LED; // LED1 (Red LED)
const int led2 = GREEN_LED; // LED2 (Green LED)

void setup() {
    pinMode(led1, OUTPUT); // output direction
    pinMode(led2, OUTPUT); // output direction
    pinMode(button1, INPUT_PULLUP); // input direction, pull-up resistor
    pinMode(button2, INPUT_PULLUP); // input direction, pull-up resistor
    Serial.begin(9600); // UART init.
    delay(100); // delay
    Serial.println("Connected"); // send data over UART
}

void loop() {
    int buttonState; // variable
    buttonState = digitalRead(button1); // read button1
    if (buttonState == LOW) { // check button state
        digitalWrite(led1, HIGH); // turn on led1
        Serial.println("Button1 pressed"); // send data over UART
    } else digitalWrite(led1, LOW); // turn off led1

    buttonState = digitalRead(button2); // read button2
    if (buttonState == LOW) { // check button state
        digitalWrite(led2, HIGH); // turn on led2
        Serial.println("Button2 pressed"); // send data over UART
    } else digitalWrite(led2, LOW); // turn off led2
    delay(100); // delay
}
```

Program 25.1. Energia GPIO and UART

Energia Code, Blink and Button

```
const int led1 = RED_LED;
const int led2 = GREEN_LED;

void setup() {
    pinMode(led1, OUTPUT);
    pinMode(led2, OUTPUT);
}
```

```
void loop() {
    digitalWrite(led1, HIGH);
    digitalWrite(led2, HIGH);
    delay(1000);
    digitalWrite(led1, LOW);
    digitalWrite(led2, LOW);
    delay(1000);
}
```

```
const int button1 = PUSH1;
const int button2 = PUSH2;
const int led1 = RED_LED;
const int led2 = GREEN_LED;
```

```
void setup() {
    pinMode(led1, OUTPUT);
    pinMode(led2, OUTPUT);
    pinMode(button1, INPUT_PULLUP);
    pinMode(button2, INPUT_PULLUP);
}
```

```
void loop() {
    int ButtonState;
    ButtonState = digitalRead(button1);
    if (ButtonState == HIGH) digitalWrite(led1, HIGH);
    else digitalWrite(led1, LOW);
    ButtonState = digitalRead(button2);
    if (ButtonState == HIGH) digitalWrite(led2, HIGH);
    else digitalWrite(led2, LOW);
}
```

```
from sense_emu import SenseHat
from time import sleep
```

```
sense = SenseHat()
while True:
    kt=sense.get_temperature()
    kp=sense.get_pressure()
    kh=sense.get_humidity()
```

```
print("Temperature: "+str(kt)," Pressure: "+str(kp)," Humidity: "+str(kh));
sleep(1)
```

Chapter 27 – Embedded Systems Security

MSP430FR5994 Security Feature:

ASE Encryption, Software IP Protection, Crypto-bootload

JTAG pins

- TDI: Test Data In.
- TDO: Test Data Out
- TCK: Test Clock
- TMS: Test Mode Select
- Optionally, TRST: Test Reset

JTAG/Boundary Scan

- Boundary scan
- Joint Test Action Group (JTAG)
- formed in 1985
- boundary scan architecture was approved in 1990
- IEEE Std. 1149.1
- latest update was done in 2013

Lab 9,10

```
pot = pico.ADC(8)
temp = pico.ADC(4)

red = pico.Pin(15, pico.Pin.OUT)
green = pico.Pin(16, pico.Pin.OUT)
blue = pico.Pin(17, pico.Pin.OUT)

def convert_temp_f(adc_val):
    voltage = adc_val * 3.3 / 65535
    tempC = 27 - (voltage - 0.706) / 0.001721
    return tempC * 9 / 5 + 32

def set_rgb(voltage):
    if voltage < 1:
        red.value(1); green.value(1); blue.value(1)
    elif (voltage >= 1) and (voltage < 2):
        red.value(0); green.value(1); blue.value(1)
    elif (voltage >= 2) and (voltage < 3):
        red.value(1); green.value(0); blue.value(1)
    else:
        red.value(1); green.value(1); blue.value(0)

while True:
    pot_raw = pot.read_u16()
    pot_voltage = pot_raw * 3.3 / 65535
    print("ADC8 (Potentiometer):", pot_raw, "-> {:.2f} V".format(pot_voltage))

    set_rgb(pot_voltage)

    temp_raw = temp.read_u16()
    temp_f = convert_temp_f(temp_raw)
    print("ADC4 (Temperature):", temp_raw, "-> {:.2f} F".format(temp_f))

    print(".", " " * 40)
    sleep(0.5)
```

```
import machine as pico
from machine import Pin, PWM
from utime import sleep

pwm = PWM(Pin(14))
adc1 = pico.ADC(0)
mode = 1

buttons = {
    0: Pin(10, Pin.IN), # Up -> Fast
    1: Pin(11, Pin.IN), # Down -> Stop
    2: Pin(12, Pin.IN), # Left -> Slow
    3: Pin(13, Pin.IN) # Right -> PWM
}

while True:
    adc_raw = adc1.read_u16()
    for m, btn in buttons.items():
        if btn.value() == 0:
            mode = m

    pwm.duty_u16({
        0: 65535, # Fast
        1: 0, # Stop
        2: 10000, # Slow
        3: adc_raw # ADC control
    }[mode])
```

Program 24.1 – TI-RTOS GPIO Example

```
int main(void) {
    Task_Params task1Params, task2Params;
    Board_initGeneral();
    Board_initGPIO();
    GPIO_write(Board_LED0, Board_LED_OFF); // LED0 (low)
    GPIO_write(Board_LED1, Board_LED_OFF); // LED1 (low)
    Task_Params_init(&task1Params); // task1Params init.
    task1Params.arg0 = 100; // arg0, 100
    task1Params.stackSize = TASKSTACKSIZE;
    task1Params.stack = &task1Stack;
    task1Params.priority = 1; // priority, 1
    Task_construct(&task1Struct, (Task_FuncPtr)task1Fxn, &task1Params, NULL);
    // task1 creation

    Task_Params_init(&task2Params); // task2Params init.
    task2Params.arg0 = 100; // arg0, 100
    task2Params.stackSize = TASKSTACKSIZE;
    task2Params.stack = &task2Stack;
    task2Params.priority = 1; // priority, 1
    Task_construct(&task2Struct, (Task_FuncPtr)task2Fxn, &task2Params, NULL);
    // task2 creation

    BIOS_start(); // start BIOS
    return 0;
}
```